

->**Direct Addressing: Distinct array position for every possible key**

->Hashing is a technique to storing and retrieving data in $O(1)$ time. ->A technique for direct search is hashing because address is known

->It is a searching technique which is independent of number n (n ->number of record) because here we are not sequentially search the record.

->It is the process of converting a given key into another value. A **hash function** is used to generate the new value according to a mathematical algorithm. The result of a hash function is known a **hash**.

->**uniform hashing:** any given element is equally likely to hash into any of the slots available in the array.

->**Application:** Blockchain, Cryptography (Hashing enables people to get data authorization without knowing the content on the data.)

-> **Hash table** is a structure that maps keys to values.

-> **The hash function calculation has two main criteria:**

- The function H should be quick and easy to compute.
- It should uniformly distribute the hash addresses throughout the set L so that there are minimum number of collisions

Hash Function Problems:

- Keys may not be numeric.
- Number of possible keys is much larger than the space available in table.
- Different keys may map into same location
 - Hash function is not one-to-one => collision.
 - If there are too many collisions, the performance of the hash table will suffer dramatically.

-> **Division Method:** The division method involves mapping a key k into one of m slots by taking the remainder of k divided by m as expressed in the hash function. **$H(k) = k \pmod{m}$ or $H(k) = k \pmod{m} + 1$**

Where $k \pmod{m}$ denotes the remainder when k is divided by m . The second formula is used when we want hash addresses to range from 1 to m rather than from 0 to $m-1$.

Good values for m are primes not too close to exact powers of 2.

->**Mid square Method:** In this technique, a seed value is taken and it is squared. Then, some digits from the middle are extracted. These extracted digits form a number which is taken as the new seed.

The key k is squared. Then the hash function H is defined by $H(k) = p$ Where p is obtained by deleting digits from both ends of k^2

->**Folding Method:** It breaks up a key value into precise segments that are added to form a hash value.

The key k is partitioned into number of parts $k_1, k_2, \dots, k_r$ where each part, except the last, has the same number of digits as the required address. Then the parts are added together, ignoring the last carry. That is $H(k) = k_1 + k_2 + \dots + k_r$ Where the leading-digit carries, if any, are ignored. Sometimes, the even numbered parts are each reversed before the addition.

-> **Collision:** When two or more keys can generate the same hash value(not take decision which to store in hash table). **OR** when an element is inserted, it hashes to the same value as an already inserted element, then we have a collision.

->**Collision Resolution:**

- **Separate chaining (open hashing)**
- **Open addressing (closed hashing)**

->**Separate chaining:** This method is used to make each cell of hash table point to a linked list of records that have same hash function value.

Advantages:

- Better space utilization for large items.
- Simple collision handling: searching linked list.
- Overflow: we can store more items than the hash table size.
- Deletion is quick and easy: deletion from the linked list.

Disadvantages:

- Cache performance of chaining is not good as keys are stored using a linked list. ...
- Wastage of Space (Some Parts of hash table are never used)
- If the chain becomes long, then search time can become $O(n)$ in the worst case.
- Uses extra space for links.

-> **Open addressing:** A method for handling collisions. In this, all elements are stored in the hash table itself. So, at any point, the size of the table must be greater than or equal to the total number of keys (Note that we can increase table size by copying old data if needed).

Open Addressing is done in the following ways:

In **linear probing**,

- When collision occurs, we linearly probe for the next bucket.
- We keep probing until an empty bucket is found.

In **quadratic probing**,

- When collision occurs, we probe for i^2 th bucket in i^{th} iteration.
- We keep probing until an empty bucket is found.

In **double hashing**,

- We use another hash function $\text{hash2}(x)$ and look for $i * \text{hash2}(x)$ bucket in i^{th} iteration.
- It requires more computation time as two hash functions need to be computed.

In double hashing, there are *two* hash functions. The second hash function is used to provide an *offset* value in case the first function causes a collision.

- The advantage of Double hashing is that it is one of the best form of probing, producing a uniform distribution of records throughout a hash table.
- This technique does not yield any clusters

Graph

->A graph is a non-linear data structure that consists of finite set of vertices (nodes) and a set of edges connecting them.

Application of graph:

Facebook, WWW, Google maps, Flight network, Product Recommendations

->**Connected / Disconnected graph:** A graph is said to be connected if there exists at least one path between every pair of vertices. Otherwise, it is disconnected.

Directed / Undirected graph: In directed graph, each edge has a direction which determines the traversal order. In an undirected graph the edges are unidirectional, with no direction associated with them. Hence, the graph can be traversed in either direction.

->**Weighted graph:** A graph where each edge has a numerical weight assigned to it.

->**Cyclic graph** - A directed graph containing at least one graph cycle. A cyclic graph possessing exactly one (undirected, simple) cycle is called a unicyclic graph.

->**Acyclic graph (DAG)** - A directed graph that is not cyclic.

->**Dense / Sparse graph:** Dense graph is a graph in which the number of edges is close to the maximal number of edges. Sparse graph is a graph in which the number of edges is close to the minimal number of edges.

->**Simple graph** - A graph without loops and without multiple edges between two nodes.

->**Complete graph** - A undirected graph in which every pair of distinct vertices is connected by a unique edge. Every complete graph is also a simple graph.

->**Strongly Connected graph:** A directed graph is called strongly connected if there is a path in each direction between each pair of vertices of the graph.

->A Graph is said to be **labeled** if its edges are assigned a data.

->**Following are the most commonly used representations of a graph:**

- Adjacency Matrix (**Sequential representation of Graphs**)
- Adjacency List (**Linked Representation of Graph**)
- Incidence Matrix

Adjacency Matrix

- An Adjacency Matrix $A[V][V]$ is a 2D array of size $V \times V$ where V is the number of vertices in a graph.
- In this matrix, both rows and columns represent vertices. This matrix is filled with either 1 (or weight) or 0.
- 1 (or weight) represents that there is an edge from row vertex to column vertex • 0 represents that there is no edge from row vertex to column vertex.

Advantages:

- Easy to understand and implement.
- Adding/Removing an edge takes $O(1)$ time.
- Queries like whether there is an edge from vertex 'u' to vertex 'v' are efficient and can be done $O(1)$.

Disadvantages

- Consumes more space $O(V^2)$. Even if the graph is sparse, it consumes the same space.
- Adding/Removing a vertex (node) is $O(V^2)$ time because there may be many changes in the matrix

Adjacency List: An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbours of a vertex in the graph.

Advantages:

- Saves space. $O(|V|+|E|)$
- Adding an edge takes $O(1)$ time
- Adding a vertex takes $O(1)$ time

Disadvantages:

- Queries like whether there is an edge from vertex u to vertex v are not efficient and can be done $O(V)$.
- Time taken to remove an edge takes $O(E)$ time.
- Time taken to remove a vertex $O(V+E)$ time.

Incidence matrix:

- An incidence matrix is a matrix where each column represents an edge connected to two vertices.
- In this representation, the graph is represented using a matrix of $V * E$.
- Possible values for undirected graph: 0, 1.
- Possible values for directed graph: 0, 1, -1

Graph Traversal: Technique used for visiting each vertex of the graph.

Why?

- Finding all reachable nodes (for garbage collection)
- Finding the best reachable node (single-player game search) or the minmax best reachable node (two-player game search) .
- Finding the best path through a graph (for routing and map directions)

Classification: Depending on the order in which vertices are visited.

- BFS-Breadth First Search
- DFS-Depth First Search

BFS: BFS is a vertex-based technique for finding the shortest path in the graph. In BFS, one vertex is selected at a time when it is visited and marked then its adjacent are visited and stored in the queue.

Start traversal from a node and explore graph layer wise.

Which Data Structure to use? -> FIFO data structure (Queue)

How to avoid cycles? -> Keep track of visited nodes

Algorithm:

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a queue.
- **Rule 2** – If no adjacent vertex is found, remove the first vertex from the queue.
- **Rule 3** – Repeat Rule 1 and Rule 2 until the queue is empty.

Application: Peer to peer networks, Social Networking Websites, GPS Navigation System, Path finding, Broadcasting Network

DFS: The depth-first search (DFS) algorithm starts with the initial node of graph G and goes deeper until we find the goal node or the node with no children.

Which Data Structure to use? -> LIFO data structure (Stack)

How to avoid cycles? -> Keep track of visited nodes

Algorithm:

The step-by-step process to implement the DFS traversal is given as follows -

1. First, create a stack with the total number of vertices in the graph.
2. Now, choose any vertex as the starting point of traversal, and push that vertex into the stack.
3. After that, push a non-visited vertex (adjacent to the vertex on the top of the stack) to the top of the stack.
4. Now, repeat steps 3 and 4 until no vertices are left to visit from the vertex on the stack's top.
5. If no vertex is left, go back and pop a vertex from the stack.
6. Repeat steps 2, 3, and 4 until the stack is empty.

Application: Topological sorting, Scheduling problems, Cycle detection in graphs, solving puzzle with only one solution e.g., maze

->Spanning trees: A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected.

What are spanning trees used for?

Applications. Minimum spanning trees have direct applications in the design of networks, including computer networks, telecommunications networks, transportation networks, water supply networks, and electrical grids.

->Network flow problems: In graph theory, a flow network (also known as a transportation network) is a directed graph where each edge has a capacity and each edge receives a flow. The amount of flow on an edge cannot exceed the capacity of the edge.

->Warshall's algorithm for shortest path: Warshall algorithm is used to find all pair shortest path problem from a given weighted graph. As a result of this algorithm, it will generate a matrix, which will represent the minimum distance from any node to all other nodes in the graph.

->Topological sort: Topological sorting for Directed Acyclic Graph (DAG) is a linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. Topological Sorting for a graph is not possible if the graph is not a DAG. Topological Sorting can be done by both DFS as well as BFS.

What is topological sort used for?

Scheduling jobs from given dependencies among Jobs. For example, if some job requires the dependency of some other job, then we can use topological sorting.